



Projet Intégré — Systèmes Multi-Agents

Product Review Intelligence

Analyse de sentiment multi-agents avec DistilBERT fine-tuné

CrewAI · DistilBERT · FastAPI · SerpApi · Ollama

Aouami Salma

Student ID : 127443

Zineb Arrami

Student ID : 118789

Hiba Arbaoui

Student ID : 120401

Ayyadi Marwa

Student ID : 120278

Encadrée par :

Pr. Hasna El Haji

Module :

Deep Learning

Année universitaire : 2025 – 2026

Université Internationale de Rabat — ESIN 4^e année

Table des matières

1	Introduction	3
2	Évolution de nos choix techniques	4
2.1	Le backend LLM : trois tentatives	4
2.1.1	Groq — des limites de débit trop strictes	4
2.1.2	Gemini — une clé API qui ne tenait pas	4
2.1.3	Ollama — la solution qu’on a gardée	5
2.2	L’outil de recherche web : DuckDuckGo puis SerpApi	5
2.2.1	DuckDuckGo — aucun résultat exploitable	5
2.2.2	SerpApi — la solution retenue	6
3	Architecture du système	6
3.1	Vue d’ensemble	6
3.2	Les trois agents	7
3.2.1	Sentiment Analyst	7
3.2.2	Market Researcher	8
3.2.3	Report Generator	8
3.3	Orchestration et point de validation humaine	8
4	Modèle de deep learning	8
4.1	Le dataset	8
4.2	Le modèle et son entraînement	9
4.3	Les résultats	10
4.4	Intégration dans le pipeline	11
5	Interface utilisateur	12
5.1	Description	12
5.2	Captures d’écran	13
6	Gestion des erreurs et journalisation	16
6.1	Gestion des erreurs	16
6.2	Journalisation	17

7 Tests et évaluation	17
7.1 Tests de bout en bout	18
7.2 Limites identifiées	18
8 Conclusion	18

1 Introduction

Pour ce projet de fin de semestre, nous avons retenu le domaine B, **Product Review Intelligence**. Le principe tient en une phrase : on donne un nom de produit au système, il part chercher des avis en ligne, il en analyse le sentiment avec un modèle que nous avons nous-mêmes entraîné, puis il rédige un rapport de marché. Le tout se fait automatiquement, grâce à plusieurs agents qui se passent le relais.

Si nous avons choisi ce sujet, c’est parce qu’il est concret et que le résultat se voit tout de suite. On tape “iPhone 15” ou “Miss Dior”, et quelques minutes plus tard on obtient une analyse complète : le sentiment des clients, les points forts, les points faibles, et des recommandations.

Le système repose sur trois briques : un modèle DistilBERT fine-tuné pour le sentiment, SerpApi pour aller chercher les avis sur le web, et CrewAI pour faire travailler ensemble trois agents spécialisés.

Côté cahier des charges, nous avons coché toutes les cases : deux agents spécialistes et un troisième chargé de la synthèse, un modèle de deep learning entraîné par l’équipe et intégré comme un vrai outil fonctionnel, un point de validation humaine avant chaque analyse, et une journalisation de chaque action. Le tableau ci-dessous résume les choix techniques et la raison de chacun.

Composant	Technologie	Pourquoi ce choix
Agents	CrewAI	Orchestration séquentielle, agents déclaratifs
LLM	Ollama (local)	Pas de limite d’API, fonctionne hors ligne
Modèle DL	DistilBERT	Rapide, léger, efficace en classification de texte
Backend	FastAPI	Simple à déployer, asynchrone
Recherche	SerpApi	Résultats Google fiables, au format JSON
Frontend	HTML/CSS/JS	Aucune dépendance, déployable tel quel

Tableau 1: Stack technologique retenue

2 Évolution de nos choix techniques

La version finale n'est pas celle par laquelle nous avons commencé. Plusieurs outils qui semblaient parfaits sur le papier nous ont lâchés à l'usage. Plutôt que de faire comme s'ils n'avaient jamais existé, on préfère raconter ces essais : ils expliquent pourquoi la stack finale ressemble à ça.

2.1 Le backend LLM : trois tentatives

2.1.1 Groq — des limites de débit trop strictes

Notre premier choix a été **Groq**, qui propose une API gratuite pour LLaMA-3.3-70B. Les premiers essais étaient très rapides. Le souci est apparu dès qu'on a lancé le pipeline en entier : les trois agents enchaînent plusieurs appels LLM, et Groq se bloquait presque aussitôt avec des erreurs 429 `RateLimitError`.

Listing 1: Tentative de contournement avec un mécanisme de retry

```
1 for attempt in range(3):
2     try:
3         result = crew.kickoff()
4         break
5     except Exception as e:
6         if "rate_limit" in str(e).lower():
7             wait = 30 * (attempt + 1) # 30s, 60s, 90s
8             time.sleep(wait)
9         else:
10            raise e
```

Même avec ce retry, les coupures revenaient trop souvent. Une simple analyse pouvait rester bloquée plusieurs minutes à cause des quotas du tier gratuit. Inutilisable pour une démo.

2.1.2 Gemini — une clé API qui ne tenait pas

On est ensuite passés à **Gemini**, via Google AI Studio. Là, le problème était autre : la clé gratuite expirait très vite, parfois en plein milieu d'une démonstration. Il fallait la régénérer à la main, ce qui n'était pas tenable non plus.

2.1.3 Ollama — la solution qu'on a gardée

Ollama fait tourner le LLM directement en local, sans passer par une API externe. Résultat : aucune limite de débit, c'est gratuit, et c'est reproductible à l'identique d'une machine à l'autre. La configuration tient en deux lignes.

Listing 2: Configuration d'Ollama dans CrewAI

```
1 from crewai import LLM
2 llm = LLM(model="ollama/llama3", base_url="http://localhost:11434")
```

Le revers de la médaille, c'est le temps d'exécution. Le modèle tourne sur le CPU/GPU local, sans les optimisations matérielles de Groq, donc une analyse complète prend plusieurs minutes. Acceptable pour une démo, mais clairement pas pour de la production.

Solution	Gratuit	Stable	Rapide	Décision
Groq	✓	×	✓	Abandonné
Gemini	✓	×	✓	Abandonné
Ollama	✓	✓	×	Retenu

Tableau 2: Comparaison des backends LLM testés

2.2 L'outil de recherche web : DuckDuckGo puis SerpApi

2.2.1 DuckDuckGo — aucun résultat exploitable

Pour aller chercher les avis, on a d'abord tenté **duckduckgo-search** (DDGS), séduisant parce qu'il ne demande aucune clé API.

Listing 3: Code DuckDuckGo (qui n'a pas fonctionné)

```
1 from duckduckgo_search import DDGS
2
3 with DDGS() as ddgs:
4     results = ddgs.text(f"{product} customer reviews", max_results=5)
```

En pratique, les requêtes ne renvoyaient rien d'utilisable sur les produits qu'on testait. Et par-dessus, une incompatibilité de version bloquait carrément l'exécution, comme le montre la trace ci-dessous (l'erreur s'est répétée trois fois).

Listing 4: Erreur DDGS dans agent_log.json

```

1 {"timestamp": "2026-05-18T01:32:22.061105",
2   "agent": "Orchestrator", "status": "error",
3   "result": "DDGS.text() got an unexpected keyword argument 'timeout'"}

```

2.2.2 SerpApi — la solution retenue

SerpApi donne accès aux résultats de Google Search au format JSON. Les réponses sont stables et bien structurées. On a remplacé DDGS par SerpApi, et le problème a disparu immédiatement. La clé API payante était le seul vrai inconvénient, mais la fiabilité valait largement le coup.

Solution	Sans clé API	Résultats fiables	Décision
DuckDuckGo	✓	×	Abandonné
SerpApi	×	✓	Retenu

Tableau 3: Comparaison des outils de recherche

3 Architecture du système

3.1 Vue d'ensemble

Le système tient en trois couches. Le **frontend** (HTML/CSS/JS) gère l'interface et le point de validation humaine. Le **backend** (FastAPI) reçoit la requête, va chercher les avis via SerpApi, puis lance le pipeline. La **couche agents** (CrewAI) exécute les trois tâches l'une après l'autre.

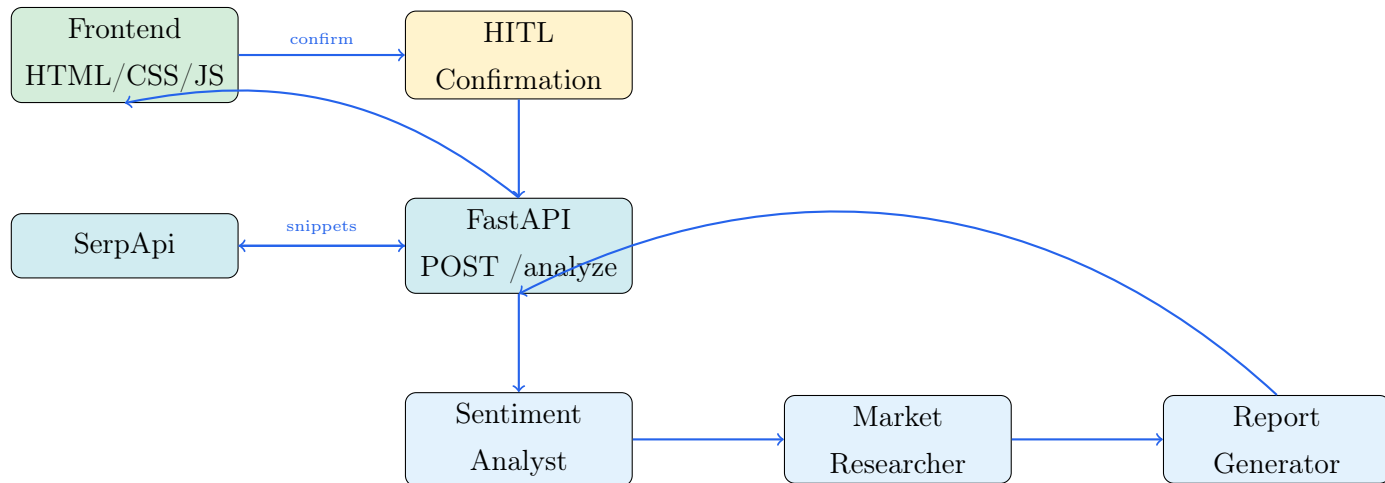


Figure 1: Flux d'exécution du système

Une fois que l'utilisateur valide, le backend récupère jusqu'à 8 snippets Google et garde les 3 plus longs. Ces avis partent ensuite dans le pipeline CrewAI, qui s'exécute en trois étapes.

3.2 Les trois agents

3.2.1 Sentiment Analyst

C'est l'agent au cœur du système. Il reçoit les avis séparés par un |, appelle notre modèle DistilBERT fine-tuné et classe chaque avis en négatif, neutre ou positif. En sortie, il renvoie un JSON avec la répartition et les scores de confiance.

Listing 5: Définition de l'agent Sentiment Analyst

```

1 sentiment_agent = Agent(
2     role="Sentiment Analyst",
3     goal="Analyser le sentiment des avis clients via
4         sentiment_analysis_tool.",
5     backstory="Expert NLP utilisant DistilBERT fine-tune sur Amazon
6         Reviews.",
7     tools=[sentiment_analysis_tool],
8     llm=llm,
9     verbose=True,
10    max_iter=4,
11 )

```

3.2.2 Market Researcher

Cet agent lance une recherche SerpApi avec la requête "{produit} customer reviews pros cons" et remonte les cinq premiers snippets. Il ne touche pas au modèle DL : son seul outil est `web_search_reviews_tool`.

3.2.3 Report Generator

Le dernier agent n'a aucun outil externe. Il reçoit le travail des deux agents précédents via le paramètre `context` de CrewAI, et rédige un rapport en cinq sections : Résumé, Sentiment, Marché, Recommandations et Conclusion.

3.3 Orchestration et point de validation humaine

Les tâches s'enchaînent en mode `Process.sequential`. Le point HITL est géré côté frontend avec `window.confirm()` : tant que l'utilisateur ne valide pas, aucun appel API n'est déclenché.

Listing 6: Lancement du pipeline CrewAI

```
1 crew = Crew(  
2     agents=[sentiment_agent, market_agent, report_agent],  
3     tasks=[task_sentiment, task_research, task_report],  
4     process=Process.sequential,  
5     verbose=False,  
6 )  
7 result = crew.kickoff()
```

4 Modèle de deep learning

4.1 Le dataset

Nous avons entraîné le modèle sur le dataset **Amazon Reviews** (fichier `Reviews.csv`), en n'utilisant que deux colonnes : le texte de l'avis (`Text`) et la note (`Score`). Les notes ont été regroupées en trois classes.

Notes	Classe	Label
1-2 étoiles	0	Négatif
3 étoiles	1	Neutre
4-5 étoiles	2	Positif

Tableau 4: Conversion des notes en classes

Le dataset d'origine est très déséquilibré : il y a beaucoup plus d'avis tranchés que d'avis neutres. Pour éviter que le modèle apprenne juste à prédire la classe majoritaire, on a sous-échantillonné pour garder exactement 2 487 exemples par classe, soit 7 461 au total. La division train/test est de 80/20, ce qui donne 5 968 exemples d'entraînement et 1 493 de test.

4.2 Le modèle et son entraînement

Nous avons pris **DistilBERT** (`distilbert-base-uncased`) comme modèle de base. C'est une version allégée de BERT — 66 M de paramètres contre 110 M — qui garde des performances proches tout en étant nettement plus rapide. On a ajouté par-dessus l'encodeur une tête de classification à 3 sorties.

L'entraînement s'est fait sur Google Colab (GPU T4) avec l'API **Trainer** de Hugging-Face. Avec seulement 2 epochs, il s'est bouclé en 68,3 secondes.

Paramètre	Valeur
Epochs	2
Batch size (train)	16
Batch size (éval)	32
Max tokens	128
Précision	FP16
Loss finale	0,614

Tableau 5: Hyperparamètres d'entraînement

Listing 7: Chargement du modèle au démarrage du serveur

```

1 tokenizer = DistilBertTokenizer.from_pretrained("
    sentiment_model_3classes")
2 dl_model = DistilBertForSequenceClassification.from_pretrained(
```

```

3         "sentiment_model_3classes")
4 dl_model.eval()

```

4.3 Les résultats

Le modèle atteint **73 % d'accuracy** sur le jeu de test. Voici le détail par classe.

Classe	Précision	Rappel	F1	Support
Négatif	0,81	0,66	0,73	505
Neutre	0,61	0,72	0,66	489
Positif	0,81	0,82	0,82	499
Macro avg	0,75	0,73	0,74	1 493

Tableau 6: Rapport de classification sur le jeu de test

La matrice de confusion donne une lecture plus parlante de ces chiffres.

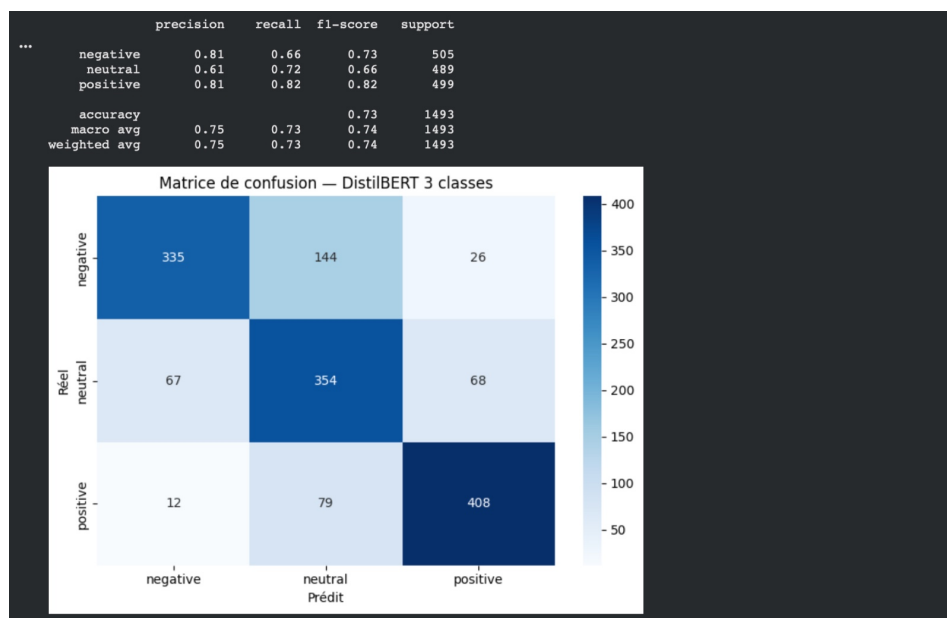


Figure 2: Rapport de classification et matrice de confusion du modèle DistilBERT à 3 classes

Ce qui ressort, c'est que les erreurs ne tombent presque jamais d'une extrémité à l'autre : seulement 26 avis négatifs ont été pris pour des positifs, et 12 positifs pour des négatifs. La grande majorité des confusions se fait avec la classe neutre au milieu — 144 négatifs et

79 positifs classés à tort comme neutres. Autrement dit, le modèle se trompe surtout entre classes voisines, ce qui est plutôt rassurant : il saisit bien la polarité générale, et hésite seulement sur l'intensité.

Sans surprise, la classe **neutre** est la plus dure à attraper ($F1 = 0,66$). Les avis à 3 étoiles sont souvent ambigus : ils mélangent du positif et du négatif dans le même texte, ce qui brouille le signal. C'est un problème bien connu en classification de sentiment à trois niveaux.

La classe positive, elle, obtient le meilleur score ($F1 = 0,82$). C'est logique : les avis 4–5 étoiles ont un vocabulaire fort et peu équivoque. Pour situer ces 73 % : un classifieur naïf qui prédirait toujours “positif” plafonnerait à 33 % sur ce dataset équilibré. Le gain est donc bien réel.

4.4 Intégration dans le pipeline

Le modèle est chargé une seule fois, au démarrage du serveur, et reste en mémoire. Il sert de deux manières : comme outil CrewAI appelé par le Sentiment Analyst, et directement depuis l'endpoint pour calculer les statistiques du dashboard.

Listing 8: Inférence dans l'outil CrewAI

```

1 @tool("sentiment_analysis_tool")
2 def sentiment_analysis_tool(reviews: str) -> str:
3     label_map = {0: "negative", 1: "neutral", 2: "positive"}
4     texts = [t.strip() for t in reviews.split("|") if t.strip()]
5     results = []
6     for text in texts:
7         inputs = tokenizer(text, return_tensors="pt",
8                             truncation=True, max_length=512)
9         with torch.no_grad():
10             outputs = dl_model(**inputs)
11             probs = torch.softmax(outputs.logits, dim=1)[0]
12             pred_idx = torch.argmax(probs).item()
13             results.append({
14                 "text": text[:100],
15                 "sentiment": label_map[pred_idx],
16                 "confidence": round(probs[pred_idx].item(), 3)
17             })
18     summary = {"total_reviews": len(results),

```

```
19         "breakdown": {s: [r["sentiment"] for r in results].count(
20             s)
21             for s in set(r["sentiment"] for r in
22                 results)}},
21         "details": results}
22     return json.dumps(summary, ensure_ascii=False, indent=2)
```

5 Interface utilisateur

5.1 Description

Le frontend est une seule page HTML/CSS/JS, sans aucun framework. Le design joue sur un fond dégradé bleu clair et deux cartes blanches. La première carte contient le champ de saisie du produit et le bouton “Lancer l’analyse”. Au clic, une boîte de confirmation apparaît (`window.confirm`), puis un indicateur de chargement affiche des messages qui défilent : “Analyse du sentiment...”, “Recherche marché...”, “Génération du rapport...”.

La seconde carte affiche les résultats : un dashboard avec le nombre d’avis, le pourcentage de positifs et la confiance moyenne, des barres de progression colorées pour les trois classes, la liste des avis annotés, et enfin le rapport complet écrit par le Report Generator.

5.2 Captures d'écran

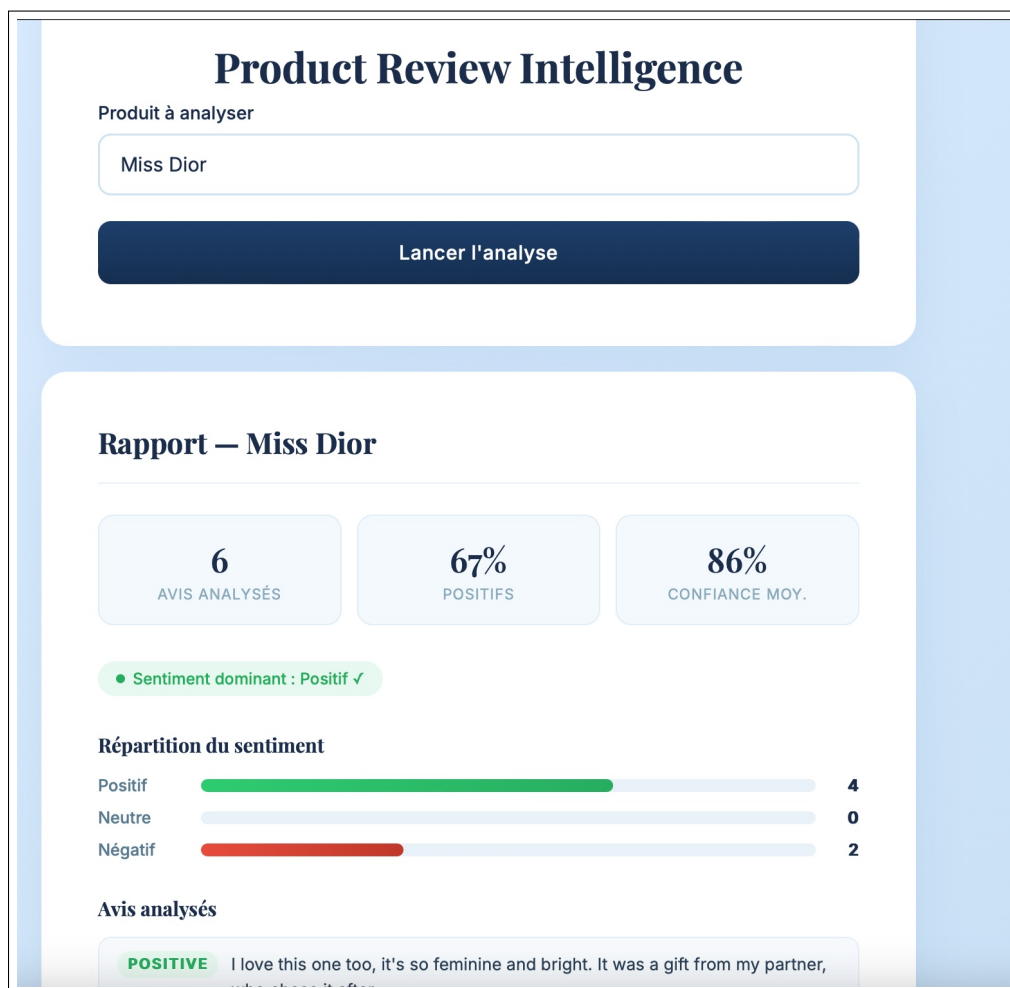


Figure 3: Formulaire de saisie et dashboard — analyse du parfum Miss Dior (6 avis, 67 % positifs, 86 % de confiance moyenne)

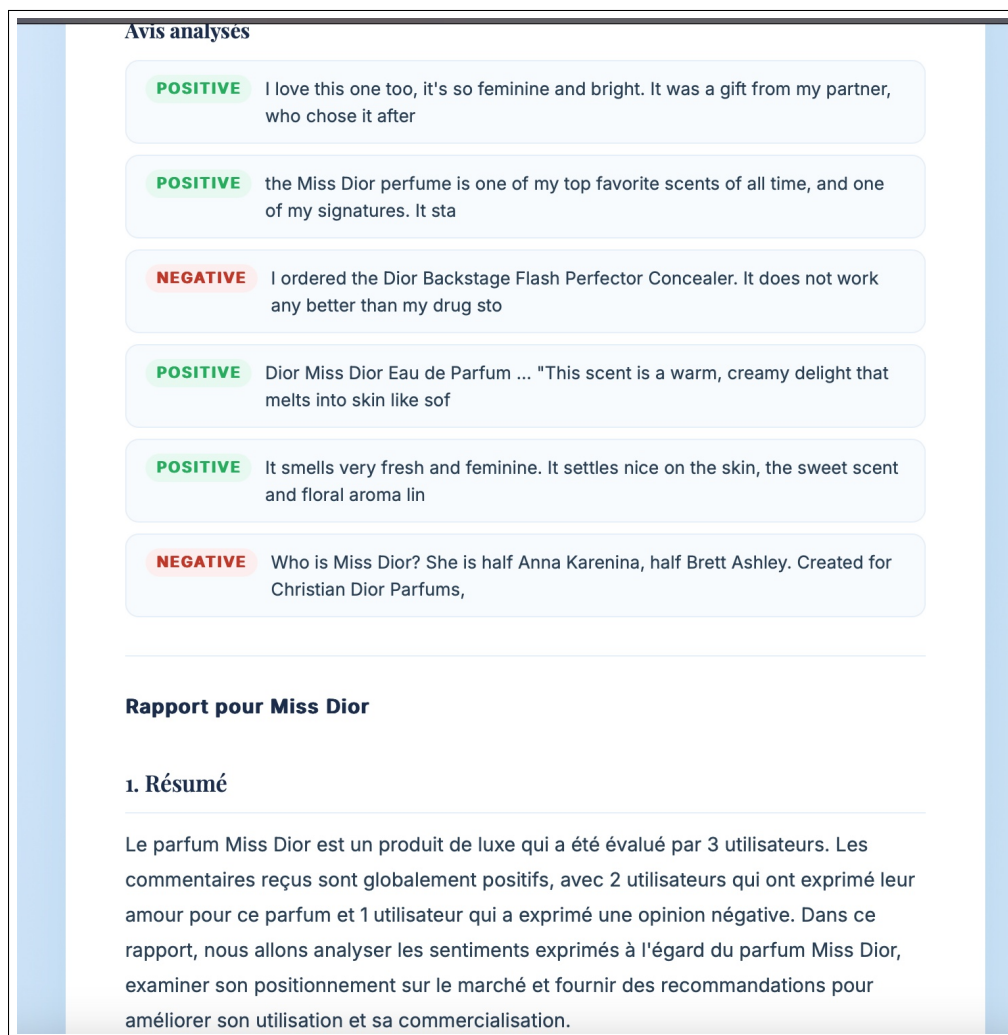


Figure 4: Avis individuels annotés par le modèle avec les labels POSITIVE et NEGATIVE

2. Sentiment

Les utilisateurs ont exprimé des sentiments variés à l'égard du parfum Miss Dior. Deux utilisateurs ont exprimé leur amour pour ce parfum, le décrivant comme féminin et lumineux, et l'un d'eux a mentionné que c'est l'un de ses parfums préférés de tous les temps. Cependant, un utilisateur a exprimé une opinion négative, mentionnant que le concealateur ne fonctionne pas mieux que d'autres produits similaires. Les commentaires suggèrent que les utilisateurs ont des opinions mitigées à l'égard du parfum Miss Dior, mais que la majorité des utilisateurs sont satisfaits de ce produit.

3. Marché

Le marché des parfums est en constante évolution, avec de nouvelles marques et de nouveaux produits qui arrivent régulièrement. Le parfum Miss Dior se positionne comme un produit de luxe, avec une image de marque forte et une réputation de qualité. Les forces du parfum Miss Dior incluent son caractère féminin et lumineux, le fait qu'il est l'un des parfums préférés de tous les temps pour certains utilisateurs, et qu'il est considéré comme un excellent choix de parfum. Cependant, les faiblesses incluent le fait que le concealateur ne fonctionne pas mieux que d'autres produits similaires, qu'il n'est pas efficace pour certaines personnes, et que le parfum peut ne pas plaire à tout le monde.

4. Recommandations

Pour améliorer l'utilisation et la commercialisation du parfum Miss Dior, nous recommandons les suivantes :

- Fournir des échantillons gratuits pour permettre aux utilisateurs de tester le parfum avant de l'acheter
- Offrir des conseils de personnalisation pour aider les utilisateurs à choisir le parfum

Figure 5: Rapport généré par le Report Generator — sections Sentiment et Marché

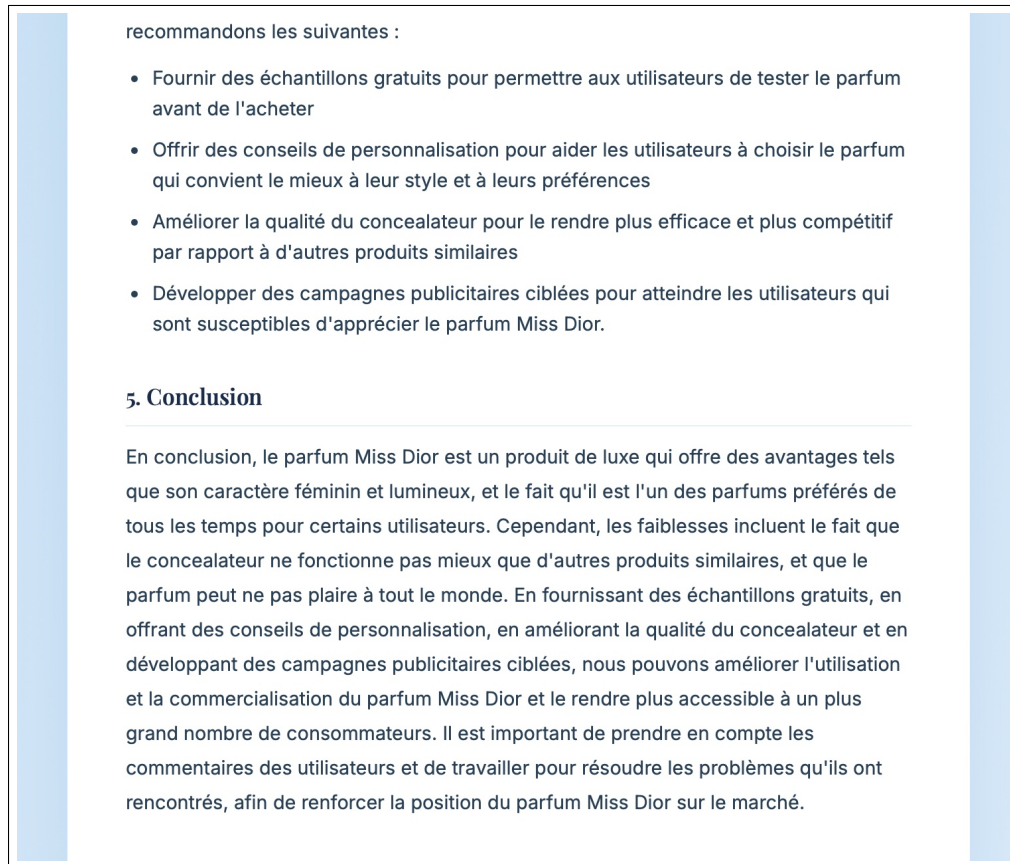


Figure 6: Suite du rapport — sections Recommandations et Conclusion

6 Gestion des erreurs et journalisation

6.1 Gestion des erreurs

Tous les appels d'outils sont entourés d'un `try/except`. En cas de pépin, l'agent reçoit un message d'erreur structuré plutôt que de voir tout le pipeline planter. L'endpoint `/analyze` vérifie aussi que SerpApi a bien renvoyé des résultats avant de lancer quoi que ce soit ; sinon, il renvoie directement une erreur au frontend.

Un dernier détail nous a donné du fil à retordre : la sortie brute du LLM contenait parfois des artefacts LaTeX (`\boxed{\}`) ou des “The final answer is:” parasites. On a ajouté un petit nettoyage par regex avant de renvoyer le rapport.

Listing 9: Nettoyage des artefacts dans la sortie du LLM

```
1 import re
2 report_text = re.sub(r'\$\boxed{\}', '', str(result))
3 report_text = re.sub(r'}\$', '', report_text)
```

```
4 report_text = re.sub(r'The final answer is:', '', report_text).strip()
```

6.2 Journalisation

Chaque action d'agent est écrite en JSON dans `agent_log.json`, avec un horodatage ISO, le nom de l'agent, l'action, le statut (`success` ou `error`) et un extrait du résultat (500 caractères au maximum).

Listing 10: Fonction de log partagée par tous les agents

```
1 def log_action(agent, action, result, status="success"):
2     entry = {
3         "timestamp": datetime.now().isoformat(),
4         "agent": agent,
5         "action": action,
6         "status": status,
7         "result": str(result)[:500]
8     }
9     logger.info(json.dumps(entry, ensure_ascii=False))
```

Voici une trace réelle, prise lors d'une analyse de l'iPhone 15.

Listing 11: Extrait de `agent_log.json`

```
1 {"timestamp": "2026-05-18T01:17:26.704254",
2   "agent": "Orchestrator", "action": "crew_kickoff",
3   "status": "success", "result": "Produit: iphone 15"}
4
5 {"timestamp": "2026-05-18T01:17:28.492728",
6   "agent": "SentimentAnalyst", "action": "analyze (5 reviews)",
7   "status": "success",
8   "result": "{\"total_reviews\": 5, \"breakdown\": {\"positive\": 5}}"}
9
10 {"timestamp": "2026-05-18T01:17:35.728446",
11   "agent": "Orchestrator", "action": "crew_finished",
12   "status": "success",
13   "result": "**Rapport d'intelligence produit : iPhone 15*...*"}

```

7 Tests et évaluation

7.1 Tests de bout en bout

On a fait tourner le système sur des produits assez différents pour vérifier qu’il tient la route dans plusieurs contextes.

Produit	Avis	Positifs	Confiance moy.	Statut
iPhone 15	5	4	93 %	✓ OK
Miss Dior	6	4	86 %	✓ OK
Dyson V15	1	1	75 %	✓ Données limitées

Tableau 7: Résultats sur différents produits

Le cas du Dyson V15 met le doigt sur un problème récurrent : quand SerpApi ne trouve pas de snippets assez longs, le pipeline n’a presque rien à se mettre sous la dent. Le système ne plante pas pour autant, mais le rapport manque forcément de matière.

7.2 Limites identifiées

Au fil des tests, on a noté plusieurs choses à améliorer. Ollama génère parfois une réponse très lente, au-delà de 5 minutes, surtout quand la machine est chargée. Sur des produits peu connus, SerpApi ne renvoie rien et le pipeline s’arrête avant même de démarrer. Et la classe neutre reste la plus difficile à classer, avec son F1 de 0,66, malgré le rééquilibrage du dataset.

Le vrai point faible, c’est le temps d’exécution avec Ollama. Pour une vraie application, il faudrait soit quantifier le modèle, soit payer un accès à une API cloud plus solide que Groq.

8 Conclusion

Ce projet nous a permis de monter un système multi-agents qui fonctionne vraiment de bout en bout, du nom de produit tapé à l’écran jusqu’au rapport final structuré. Chacun des trois agents a un rôle bien à lui, et la chaîne reste cohérente sur les différents produits qu’on a essayés.

Ce qu’on retient surtout, c’est qu’il faut tester chaque composant en conditions réelles avant de l’intégrer. Groq, Gemini, DuckDuckGo : tous les trois avaient l’air parfaits au

départ, et tous les trois nous ont lâchés à l'usage. La version finale, avec Ollama et SerpApi, est le fruit de choix pragmatiques plus que d'un plan tracé d'avance.

Côté modèle, notre DistilBERT fine-tuné atteint 73 % d'accuracy avec seulement 2 epochs sur un dataset équilibré de 7 461 exemples. Pour une tâche à trois classes, c'est un résultat solide.

Pistes d'amélioration : augmenter le volume d'entraînement pour mieux gérer la classe neutre, ajouter un cache pour ne pas relancer tout le pipeline sur un produit déjà analysé, et trouver un LLM local plus rapide qu'Ollama pour des démos plus fluides.

Conformité au cahier des charges :

- ✓ Agents : Sentiment Analyst + Market Researcher + Report Generator
- ✓ Modèle DL : DistilBERT fine-tuné, PyTorch, 73 % d'accuracy, rapport de classification
- ✓ Outils : `sentiment_analysis_tool` + `web_search_reviews_tool`
- ✓ HITL : `window.confirm()` avant chaque pipeline
- ✓ Erreurs : try/except sur tous les outils, validation SerpApi, nettoyage de la sortie LLM
- ✓ Logging : JSON horodaté dans `agent_log.json`
- ✓ Reproductibilité : modèle sauvegardé dans `sentiment_model_3classes/`